GPU Project 3: Parallel Bloom Filter with CUDA

Instructor: Dr. Yicheng Tu

Course : COP4520 Computing on Massively Parallel Systems

Deadline : 11/19 /2025 at 11:59 pm

Overview

This project focuses on implementing a parallel Bloom filter using CUDA. Students will develop a high-performance data structure that efficiently adds and checks strings in bulk using GPU acceleration, demonstrating parallel computing principles and optimization techniques.

Project Description

You will implement a parallel Bloom filter using CUDA, incorporating efficient string hashing mechanisms and GPU implementation performance. This project aims to deepen your understanding of parallel data structure design and GPU programming.Please refer the CPU code provided in the file/Project3/bloomfilter.cu

Tasks to be Performed

- Implement the SipHash algorithm for string hashing.
- Design a Bloom filter data structure with configurable false positive rates and parallel operations.
- Create GPU-accelerated functions for initializing the Bloom filter, adding strings in parallel, and checking string membership.
- Generate large sets of random strings for testing.
- Measure GPU implementation performance across different input sizes, false positive rates, and thread block configurations.

Input/Output of Program

Your program should be executed with the following command-line arguments:

/apps/GPU_course/runScript.sh proj3.cu {# of elements} {desired % error} {block size}

Where:
- `{# of elements}`: The number of elements to be added to the Bloom filter
- `{desired % error}`: The desired false positive rate (between 0 and 1)
- `{block size}`: The number of threads per block for CUDA execution

Output should include the time for generation and false negatives.

How to run the provided CPU code ?

It requires only two command line arguments which is the # of elements and desired % error
So please run the CPU code as shown below:
/apps/GPU_course/runScript.sh bloomfilter_CPU.cu 10000 0.1
Where 10000 is the # of elements and 0.1 is the desired % error .

Reference for SipHash implementation:

https://github.com/veorq/SipHash/tree/master

Please find below the expected output - this is just and example (Note that Speedup must be calculated)

[CPU] Insert+Query(or Total time of generation): 10.050 ms
[CPU] False negatives: 0/10000
[GPU] Insert+Query: 0.072 ms (139.8x speedup)
[GPU] False negatives: 0/10000

Measuring Time

Use CUDA events for precise timing measurements of GPU operations. Implement start_timer() and stop_timer() functions to facilitate accurate performance of GPU implementations.

Environment

All projects will be tested on CUDA 11.4 on one machine of the GAIVI cluster. You should have downloaded a document with the instructions to log into GAIVI. If you prefer to work on your own computer, make sure your project can be executed on the GPU card of a GAIVI node.

Instructions to Submit

You should submit one .cu file containing your implementation of the parallel Bloom filter and a separate .pdf file containing the report. Put both files in a folder, zip the folder and name the zipped file proj3-xxx.zip (or proj3-xxx.tar if you prefer using TAR for compression), where xxx is your USF NetID. Submit the zipped file only via the assignment link in Canvas. E-mail or any other form of submission will not be graded. Once you submit your file to Canvas, try to download the submitted file and open it on your machine to ensure no data transmission problems occurred. We suggest you finish the project and submit the file well before the deadline.

Grading Criteria

- All programs that fail to compile will receive zero points. We expect your submission can be compiled by running a simple command line as in previous projects.
- If any changes are needed to make the main code work after the submission deadline, they should be limited to 3 lines of code. This will incur a 30% penalty.
- The program should run for different parameters, as specified by the command-line arguments.
- We will carefully check the code related to performance measurement. Any tricks played to report shorter running time than the actual one will be treated as cheating and incur a heavy penalty.

Project Requirements

1. Bloom Filter Implementation

- Implement the **Bloom Filter entirely on the GPU** using CUDA.

- Use a hash function (**SipHash**) for efficient insertion and membership checking.

2. SipHash Implementation

- Implement **SipHash** as the hash function.
- This will be used to hash strings and map them to positions in the Bloom filter's bit array.

3. Dynamic String Generation

- Generate random **alphanumeric strings of varying lengths (5–20 characters)** dynamically during each run.
- Insert these strings into the Bloom filter and then check their membership.
- Strings must be generated at runtime (not pre-stored).

4. Performance Measurement

- Use **CUDA events** to measure the time spent on Bloom filter operations (insertions + membership checks).
- Exclude string generation time from performance measurements.
- The workflow should be: generate strings → insert into Bloom filter → check membership.